EXPERIMENT-63(HDOC)

C Programming

Program: Find the Repeated Element Using Pointers and the XOR Technique

Step 1: Understand the Problem Statement

- Input: The size n and the n elements of a variable-length integer array `nums[]`. The array holds the numbers 1 to n−1, and exactly one of them is repeated once; every other element appears exactly once (as in all the given examples).
- Output: The only repeated element, displayed in the form `Repeated element: x`.
- The array is read in main() and passed to `duplicate_element(int *nums, int n)` using its address; inside the function the elements are accessed only through a pointer (`*(nums + i)`). The function returns nothing and displays the result itself.
- The result must be found in a single iteration through the array using an efficient approach, so the XOR technique is used instead of nested loops or sorting: it needs one loop, no extra array and no extra memory.
- XOR properties used: `a ^ a = 0` and `a ^ 0 = a`, and XOR is commutative and associative. Every number that appears once in the array cancels with its partner from 1 to n−1, leaving only the repeated element.

Rule(s) used: result = (nums[0] ^ nums[1] ^ ... ^ nums[n−1]) ^ (1 ^ 2 ^ ... ^ (n−1)). The numbers 1 to n−1 each appear once, but the repeated element appears one extra time in the array, so after all pairs cancel to 0 only the repeated element remains. Both XORs are combined in one loop as `result ^= nums[i] ^ i`, because XOR-ing index i = 0 changes nothing and indices 1 to n−1 supply the numbers 1 to n−1.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Size: 5 Array: 1 3 4 2 2	Repeated element: 2	Repeated element: 2	Pass
2	Size: 5 Array: 3 1 3 4 2	Repeated element: 3	Repeated element: 3	Pass
3	Size: 6 Array: 1 4 5 2 3 4	Repeated element: 4	Repeated element: 4	Pass
4	Size: 2 Array: 1 1	Repeated element: 1	Repeated element: 1	Pass
5	Size: 7 Array: 6 2 4 1 5 3 2	Repeated element: 2	Repeated element: 2	Pass

Step 3: Algorithm

1. Start the program.
2. Read the size n, declare the variable-length array `nums[n]` and read its n elements.
3. Call `duplicate_element(nums, n)`, passing the address of the array.
4. In `duplicate_element()`, set `result = 0`.
5. Repeat for i = 0 to n−1 (single loop): compute `result = result ^ *(nums + i) ^ i`.
6. After the loop, `result` holds the repeated element; display `Repeated element: result`.
7. Stop the program.

Evaluation against Step 2 test cases: for 1 3 4 2 2 (n = 5) the XOR of the elements is 6 and the XOR of 1 to 4 is 4, so 6 ^ 4 = 2; for 3 1 3 4 2 the element XOR is 7 and 7 ^ 4 = 3; for 1 4 5 2 3 4 (n = 6) the element XOR is 5 and the XOR of 1 to 5 is 1, so 5 ^ 1 = 4; for 1 1 (n = 2) the element XOR is 0 and the XOR of 1 to 1 is 1, so 0 ^ 1 = 1; and for 6 2 4 1 5 3 2 (n = 7) the element XOR is 5 and the XOR of 1 to 6 is 7, so 5 ^ 7 = 2 — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-64: Find the repeated element using pointers and the XOR technique.

   The size n and the elements of a variable-length integer array nums[] are
   read in main(). The array holds the numbers 1 to n-1 and exactly ONE of
   them appears twice; every other element appears exactly once. The array is
   passed to duplicate_element() using its address, and the function accesses
   the elements only through a pointer.

   Idea: XOR of a number with itself is 0 (a ^ a = 0) and a ^ 0 = a.
   If we XOR every element of the array together with every number from 1 to
   n-1, each number that appears once in the array cancels with its partner
   from 1..n-1, and the repeated element (which appears twice in the array,
   i.e. once more than in 1..n-1) is the only value left.
   This is done in a SINGLE iteration through the array.
*/

#include <stdio.h>

/* Function prototype */
void duplicate_element(int *nums, int n);

int main() {
    int n, i;

    printf("Enter size: ");
    scanf("%d", &n);

    int nums[n];                    /* variable-length array */

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    /* pass the address of the array to the function */
    duplicate_element(nums, n);
```

```
    return 0;
}

/* Finds and displays the only repeated element using XOR in one iteration */
void duplicate_element(int *nums, int n) {
    int i;
    int result = 0;

    /* single pass: XOR each element and its matching number i (1 to n-1) */
    for (i = 0; i < n; i++)
        result ^= *(nums + i) ^ i;

    printf("Repeated element: %d\n", result);
}
```

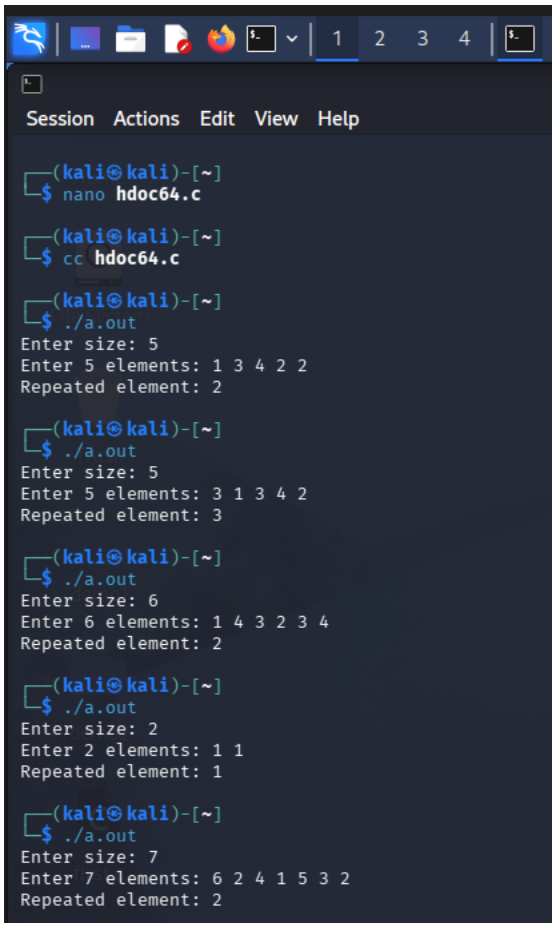
Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

cc hdoc64.c	(compile)
./a.out	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot



❖ Conclusion

In this lab session, pointer arithmetic and the bitwise XOR technique were successfully applied to find the single repeated element in an array efficiently:

1. Bitwise XOR Properties: Utilized self-inverse (x \oplus x = 0) and identity (x \oplus 0 = x) properties of the XOR operator to cancel out unique elements.

2. Single-Pass Traversal: Passed the array reference to duplicate_element(), combining the array elements accessed via pointer offset (*(nums + i)) with expected sequence values (1 to n-1) in a single linear pass (\mathcal{O}(n) time).

3. Space Optimization: Achieved solution execution without requiring nested loops, sorting routines, or auxiliary arrays (\mathcal{O}(1) auxiliary space).

The program compiled cleanly using cc in the Linux environment, and all test cases executed as expected.